

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.useradminRe-`
lease 1.1.2-SNAPSHOT

September 2026



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
107	User Admin Service Specification	7
107.1	Introduction	7
107.2	Authentication	9
107.3	Authorization	11
107.4	Repository Maintenance	13
107.5	User Admin Events	13
107.6	Security	14
107.7	Relation to JAAS	15
107.8	org.osgi.service.useradmin	16
107.9	References	26

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

107 User Admin Service Specification

Version 1.1

107.1 Introduction

OSGi frameworks are often used in places where end users or devices initiate actions. These kinds of actions inevitably create a need for authenticating the initiator. Authenticating can be done in many different ways, including with passwords, one-time token cards, biometrics, and certificates.

Once the initiator is authenticated, it is necessary to verify that this principal is authorized to perform the requested action. This authorization can only be decided by the operator of the OSGi environment, and thus requires administration.

The User Admin service provides this type of functionality. Bundles can use the User Admin service to authenticate an initiator and represent this authentication as an Authorization object. Bundles that execute actions on behalf of this user can use the Authorization object to verify if that user is authorized.

The User Admin service provides authorization based on who runs the code, instead of using the Java code-based permission model. See [1] *The Java Security Architecture for JDK 1.2* . It performs a role similar to [2] *Java Authentication and Authorization Service* .

107.1.1 Essentials

- *Authentication* - A large number of authentication schemes already exist, and more will be developed. The User Admin service must be flexible enough to adapt to the many different authentication schemes that can be run on a computer system.
- *Authorization* - All bundles should use the User Admin service to authenticate users and to find out if those users are authorized. It is therefore paramount that a bundle can find out authorization information with little effort.
- *Security* - Detailed security, based on the Framework security model, is needed to provide safe access to the User Admin service. It should allow limited access to the credentials and other properties.
- *Extensibility* - Other bundles should be able to build on the User Admin service. It should be possible to examine the information from this service and get real-time notifications of changes.
- *Properties* - The User Admin service must maintain a persistent database of users. It must be possible to use this database to hold more information about this user.
- *Administration* - Administering authorizations for each possible action and initiator is time-consuming and error-prone. It is therefore necessary to have mechanisms to group end users and make it simple to assign authorizations to all members of a group at one time.

107.1.2 Entities

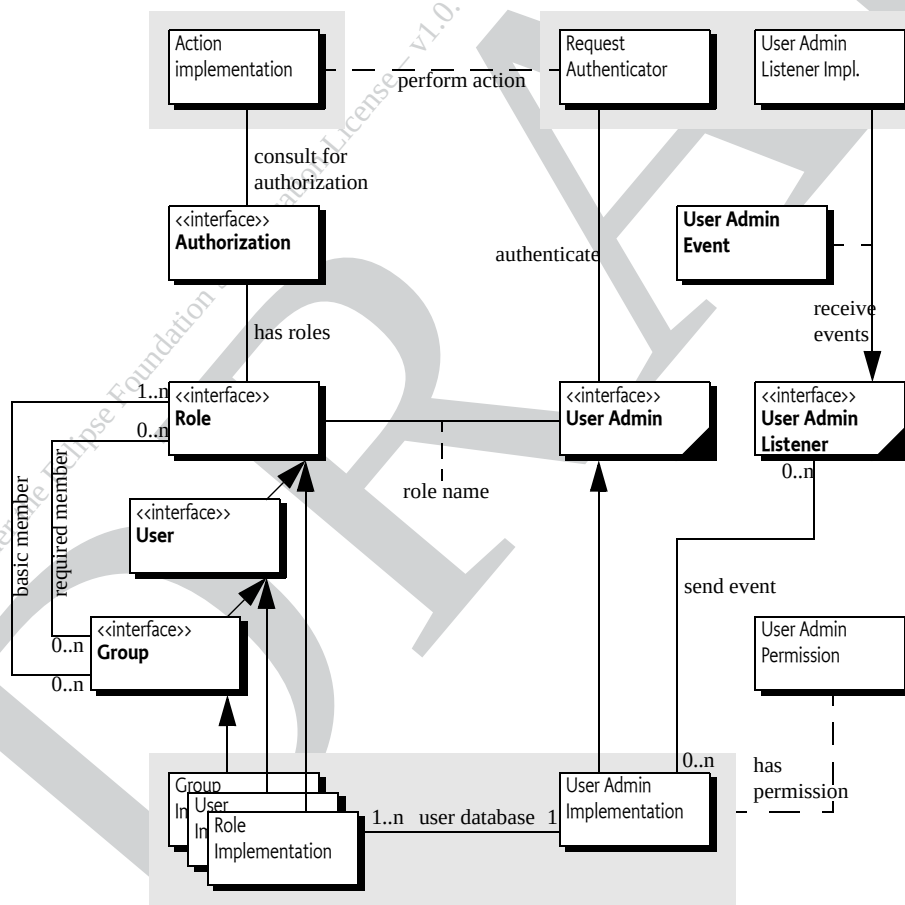
This Specification defines the following User Admin service entities:

- *User Admin* - This interface manages a database of named roles which can be used for authorization and authentication purposes.
- *Role* - This interface exposes the characteristics shared by all roles: a name, a type, and a set of properties.

- *User* - This interface (which extends Role) is used to represent any entity which may have credentials associated with it. These credentials can be used to authenticate an initiator.
- *Group* - This interface (which extends User) is used to contain an aggregation of named Role objects (Group or User objects).
- *Authorization* - This interface encapsulates an authorization context on which bundles can base authorization decisions.
- *User Admin Event* - This class is used to represent a role change event.
- *User Admin Listener* - This interface provides a listener for events of type UserAdminEvent that can be registered as a service.
- *User Admin Permission* - This permission is needed to configure and access the roles managed by a User Admin service.
- *Role.USER_ANYONE* - This is a special User object that represents *any* user, it implies all other User objects. It is also used when a Group is used with only basic members. The Role.USER_ANYONE is then the only required member.

Figure 107.1

User Admin Service, org.osgi.service.useradmin



107.1.3

Operation

An Operator uses the User Admin service to define OSGi framework users and configure them with properties, credentials, and *roles*.

A Role object represents the initiator of a request (human or otherwise). This specification defines two types of roles:

- *User* - A User object can be configured with credentials, such as a password, and properties, such as address, telephone number, and so on.
- *Group* - A Group object is an aggregation of *basic* and *required* roles. Basic and required roles are used in the authorization phase.

An OSGi framework can have several entry points, each of which will be responsible for authenticating incoming requests. An example of an entry point is the Servlet Whiteboard Specification, which delegates authentication of incoming requests to the `handleSecurity` method of the `Servlet-ContextHelper` object that was specified when the target servlet or resource of the request was registered.

The OSGi framework entry points should use the information in the User Admin service to authenticate incoming requests, such as a password stored in the private credentials or the use of a certificate.

A bundle can determine if a request for an action is authorized by looking for a Role object that has the name of the requested action.

The bundle may execute the action if the Role object representing the initiator *implies* the Role object representing the requested action.

For example, an initiator Role object *X* implies an action Group object *A* if:

- *X* implies at least one of *A*'s basic members, and
- *X* implies all of *A*'s required members.

An initiator Role object *X* implies an action User object *A* if:

- *A* and *X* are equal.

The Authorization class handles this non-trivial logic. The User Admin service can capture the privileges of an authenticated User object into an Authorization object. The `Authorization.hasRole` method checks if the authenticated User object has (or implies) a specified action Role object.

For example, in the case of a web application, a server component like a servlet filter can authenticate the initiator and place an Authorization object in the request. The servlet calls the `hasRole` method on this Authorization object to verify that the initiator has the authority to perform a certain action.

107.2 Authentication

The authentication phase determines if the initiator is actually the one it says it is. Mechanisms to authenticate always need some information related to the user or the OSGi framework to authenticate an external user. This information can consist of the following:

- A secret known only to the initiator.
- Knowledge about cards that can generate a unique token.
- Public information like certificates of trusted signers.
- Information about the user that can be measured in a trusted way.
- Other specific information.

107.2.1 Repository

The User Admin service offers a repository of Role objects. Each Role object has a unique name and a set of properties that are readable by anyone, and are changeable when the changer has the `UserAdminPermission`. Additionally, User objects, a sub-interface of Role, also have a set of private protected

properties called credentials. Credentials are an extra set of properties that are used to authenticate users and that are protected by UserAdminPermission.

Properties are accessed with the Role.getProperties() method and credentials with the User.getCredentials() method. Both methods return a Dictionary object containing key/value pairs. The keys are String objects and the values of the Dictionary object are limited to String or byte[] objects.

This specification does not define any standard keys for the properties or credentials. The keys depend on the implementation of the authentication mechanism and are not formally defined by OSGi specifications.

The repository can be searched for objects that have a unique property (key/value pair) with the method UserAdmin.getUser(String,String). This makes it easy to find a specific user related to a specific authentication mechanism. For example, a secure card mechanism that generates unique tokens could have a serial number identifying the user. The owner of the card could be found with the method

```
User owner = useradmin.getUser(  
    "secure-card-serial", "132456712-1212" );
```

If multiple User objects have the same property (key *and* value), a null is returned.

There is a convenience method to verify that a user has a credential without actually getting the credential. This is the User.hasCredential(String,Object) method.

Access to credentials is protected on a name basis by UserAdminPermission. Because properties can be read by anyone with access to a User object, UserAdminPermission only protects change access to properties.

107.2.2

Basic Authentication

The following example shows a very simple authentication algorithm based on passwords.

The vendor of the authentication bundle uses the property "com.acme.basic-id" to contain the name of a user as it logs in. This property is used to locate the User object in the repository. Next, the credential "com.acme.password" contains the password and is compared to the entered password. If the password is correct, the User object is returned. In all other cases a SecurityException is thrown.

```
public User authenticate(  
    UserAdmin ua, String name, String pwd )  
    throws SecurityException {  
    User user = ua.getUser("com.acme.basicid",  
        username);  
    if (user == null)  
        throw new SecurityException( "No such user" );  
  
    if (!user.hasCredential("com.acme.password", pwd))  
        throw new SecurityException(  
            "Invalid password" );  
    return user;  
}
```

107.2.3

Certificates

Authentication based on certificates does not require a shared secret. Instead, a certificate contains a name, a public key, and the signature of one or more signers.

The name in the certificate can be used to locate a User object in the repository. Locating a User object, however, only identifies the initiator and does not authenticate it.

1. The first step to authenticate the initiator is to verify that it has the private key of the certificate.

2. Next, the User Admin service must verify that it has a User object with the right property, for example "com.acme.certificate"="Fudd".
3. The next step is to see if the certificate is signed by a trusted source. The bundle could use a central list of trusted signers and only accept certificates signed by those sources. Alternatively, it could require that the certificate itself is already stored in the repository under a unique key as a byte[] in the credentials.
4. In any case, once the certificate is verified, the associated User object is authenticated.

107.3 Authorization

The User Admin service authorization architecture is a *role-based model*. In this model, every action that can be performed by a bundle is associated with a *role*. Such a role is a Group object (called group from now on) from the User Admin service repository. For example, if a servlet could be used to activate the alarm system, there should be a group named AlarmSystemActivation.

The operator can administrate authorizations by populating the group with User objects (users) and other groups. Groups are used to minimize the amount of administration required. For example, it is easier to create one Administrators group and add administrative roles to it rather than individually administer all users for each role. Such a group requires only one action to remove or add a user as an administrator.

The authorization decision can now be made in two fundamentally different ways:

An initiator could be allowed to carry out an action (represented by a Group object) if it implied any of the Group object's members. For example, the AlarmSystemActivation Group object contains an Administrators and a Family Group object:

```
Administrators    = { Elmer, Pepe, Bugs }
Family           = { Elmer, Pepe, Daffy }

AlarmSystemActivation = { Administrators, Family }
```

Any of the four members Elmer, Pepe, Daffy, or Bugs can activate the alarm system.

Alternatively, an initiator could be allowed to perform an action (represented by a Group object) if it implied *all* the Group object's members. In this case, using the same AlarmSystemActivation group, only Elmer and Pepe would be authorized to activate the alarm system, since Daffy and Bugs are *not* members of *both* the Administrators and Family Group objects.

The User Admin service supports a combination of both strategies by defining both a set of *basic members* (any) and a set of *required members* (all).

```
Administrators    = { Elmer, Pepe, Bugs }
Family           = { Elmer, Pepe, Daffy }

AlarmSystemActivation
  required        = { Administrators }
  basic           = { Family }
```

The difference is made when Role objects are added to the Group object. To add a basic member, use the Group.addMember(Role) method. To add a required member, use the Group.addRequiredMember(Role) method.

Basic members define the set of members that can get access and required members reduce this set by requiring the initiator to *imply* each required member.

A User object implies a Group object if it implies the following:

- *All* of the Group's required members, and
- *At least* one of the Group's basic members

A User object always implies itself.

If only required members are used to qualify the implication, then the standard user Role.

`USER_ANYONE` can be obtained from the User Admin service and added to the Group object. This Role object is implied by anybody and therefore does not affect the required members.

107.3.1

The Authorization Object

The complexity of authorization is hidden in an Authorization class. Normally, the authenticator should retrieve an Authorization object from the User Admin service by passing the authenticated User object as an argument. This Authorization object is then passed to the bundle that performs the action. This bundle checks the authorization with the `Authorization.hasRole(String)` method. The performing bundle must pass the name of the action as an argument. The Authorization object checks whether the authenticated user implies the Role object, specifically a Group object, with the given name. This is shown in the following example.

```
public void activateAlarm(Authorization auth) {
    if ( auth.hasRole( "AlarmSystemActivation" ) ) {
        // activate the alarm
        ...
    }
    else throw new SecurityException(
        "Not authorized to activate alarm" );
}
```

107.3.2

Authorization Example

This section demonstrates a possible use of the User Admin service. The service has a flexible model and many other schemes are possible.

Assume an Operator installs an OSGi framework. Bundles in this environment have defined the following action groups:

```
AlarmSystemControl
InternetAccess
TemperatureControl
PhotoAlbumEdit
PhotoAlbumView
PortForwarding
```

Installing and uninstalling bundles could potentially extend this set. Therefore, the Operator also defines a number of groups that can be used to contain the different types of system users.

```
Administrators
Buddies
Children
Adults
Residents
```

In a particular instance, the Operator installs it in a household with the following residents and buddies:

```
Residents:    Elmer, Fudd, Marvin, Pepe
Buddies:      Daffy, Foghorn
```

First, the residents and buddies are assigned to the system user groups. Second, the user groups need to be assigned to the action groups.

The following tables show how the groups could be assigned.

Table 107.1 *Example Groups with Basic and Required Members*

Groups	Elmer	Fudd	Marvin	Pepe	Daffy	Foghorn
Residents	Basic	Basic	Basic	Basic	-	-
Buddies	-	-	-	-	Basic	Basic
Children	-	-	Basic	Basic	-	-
Adults	Basic	Basic	-	-	-	-
Administrators	Basic	-	-	-	-	-

Table 107.2 *Example Action Groups with their Basic and Required Members*

Groups	Residents	Buddies	Children	Adults	Admin
AlarmSystemControl	Basic	-	-	-	Required
InternetAccess	Basic	-	-	Required	-
TemperatureControl	Basic	-	-	Required	-
PhotoAlbumEdit	Basic	-	Basic	Basic	-
PhotoAlbumView	Basic	Basic	-	-	-
PortForwarding	Basic	-	-	-	Required

107.4 Repository Maintenance

The UserAdmin interface is a straightforward API to maintain a repository of User and Group objects. It contains methods to create new Group and User objects with the createRole(String,int) method. The method is prepared so that the same signature can be used to create new types of roles in the future. The interface also contains a method to remove a Role object.

The existing configuration can be obtained with methods that list all Role objects using a filter argument. This filter, which has the same syntax as the Framework filter, must only return the Role objects for which the filter matches the properties.

Several utility methods simplify getting User objects depending on their properties.

107.5 User Admin Events

Changes in the User Admin service can be determined in real time. Each User Admin service implementation must send a UserAdminEvent object to any service in the Framework service registry that is registered under the UserAdminListener interface. This event must be send asynchronously from the cause of the event. The way events must be delivered is the same as described in *Delivering Events* of ???.

This procedure is demonstrated in the following code sample.

```
class Listener implements UserAdminListener{
    public void roleChanged( UserAdminEvent event ) {
        ...
    }
}
public class MyActivator
```

```
implements BundleActivator {  
    public void start( BundleContext context ) {  
        context.registerService(  
            UserAdminListener.class.getName(),  
            new Listener(), null );  
    }  
    public void stop( BundleContext context ) {}  
}
```

It is not necessary to unregister the listener object when the bundle is stopped because the Framework automatically unregisters it. Once registered, the `UserAdminListener` object must be notified of all changes to the role repository.

107.5.1

Event Admin and User Admin Change Events

User Admin events must be delivered asynchronously to the Event Admin service by the implementation, if present. The topic of a User Admin Event is:

`org.osgi/service/useradmin/UserAdmin/<eventtype>`

The following event types are supported:

`ROLE_CREATED`
`ROLE_CHANGED`
`ROLE_REMOVED`

All User Admin Events must have the following properties:

- `event` - (`UserAdminEvent`) The event that was broadcast by the User Admin service.
- `role` - (`Role`) The Role object that was created, modified or removed.
- `role.name` - (`String`) The name of the role.
- `role.type` - (`Integer`) One of `ROLE`, `USER` or `GROUP`.
- `service` - (`ServiceReference`) The Service Reference of the User Admin service.
- `service.id` - (`Long`) The User Admin service's ID.
- `service.objectClass` - (`String[]`) The User Admin service's object class (which must include `org.osgi.service.useradmin.UserAdmin`)
- `service.pid` - (`String`) The User Admin service's persistent identity

107.6

Security

The User Admin service is related to the security model of the OSGi framework, but is complementary to the [1] *The Java Security Architecture for JDK 1.2*. The final permission of most code should be the intersection of the Java 2 Permissions, which are based on the code that is executing, and the User Admin service authorization, which is based on the user for whom the code runs.

107.6.1

User Admin Permission

The User Admin service defines the `UserAdminPermission` class that can be used to restrict bundles in accessing credentials. This permission class has the following actions:

- `changeProperty` - This permission is required to modify properties. The name of the permission is the prefix of the property name.
- `changeCredential` - This action permits changing credentials. The name of the permission is the prefix of the name of the credential.

- `getCredential` - This action permits getting credentials. The name of the permission is the prefix of the credential.

If the name of the permission is "admin", it allows the owner to administer the repository. No action is associated with the permission in that case.

Otherwise, the permission name is used to match the property name. This name may end with a `.*` string to indicate a wildcard. For example, `com.acme.*` matches `com.acme.fudd.elmer` and `com.acme.bugs`.

107.7 Relation to JAAS

At a glance, the Java Authorization and Authentication Service (JAAS) seems to be a very suitable model for user administration. The OSGi organization, however, decided to develop an independent User Admin service because JAAS was not deemed applicable. The reasons for this include dependency on Java SE version 1.3 ("JDK 1.3") and existing mechanisms in the previous OSGi Service Gateway 1.0 specification.

107.7.1 JDK 1.3 Dependencies

The authorization component of JAAS relies on the `java.security.DomainCombiner` interface, which provides a means to dynamically update the `ProtectionDomain` objects affiliated with an `AccessControlContext` object.

This interface was added in JDK 1.3. In the context of JAAS, the `SubjectDomainCombiner` object, which implements the `DomainCombiner` interface, is used to update `ProtectionDomain` objects. The permissions of `ProtectionDomain` objects depend on where code came from and who signed it, with permissions based on who is running the code.

Leveraging JAAS would have resulted in user-based access control on the OSGi framework being available only with JDK 1.3, which was not deemed acceptable.

107.7.2 Existing OSGi Mechanism

JAAS provides a pluggable authentication architecture, which enables applications and their underlying authentication services to remain independent from each other.

The Servlet Whiteboard Specification already provides a similar feature by allowing servlet and resource registrations to be supported by an `ServletContextHelper` object, which uses a callback mechanism to perform any required authentication checks before granting access to the servlet or resource. This way, the registering bundle has complete control on a per-servlet and per-resource basis over which authentication protocol to use, how the credentials presented by the remote requester are to be validated, and who should be granted access to the servlet or resource.

107.7.3 Future Road Map

In the future, the main barrier of 1.3 compatibility will be removed. JAAS could then be implemented in an OSGi environment. At that time, the User Admin service will still be needed and will provide complementary services in the following ways:

- The authorization component relies on group membership information to be stored and managed outside JAAS. JAAS does not manage persistent information, so the User Admin service can be a provider of group information when principals are assigned to a `Subject` object.
- The authorization component allows for credentials to be collected and verified, but a repository is needed to actually validate the credentials.

In the future, the User Admin service can act as the back-end database to JAAS. The only aspect JAAS will remove from the User Admin service is the need for the `Authorization` interface.

107.8 org.osgi.service.useradmin

User Admin Package Version 1.1.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.useradmin; version="[1.1,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.useradmin; version="[1.1,1.2)"}
```

107.8.1 Summary

- Authorization - The {@code Authorization} interface encapsulates an authorization context on which bundles can base authorization decisions, where appropriate.
- Group - A named grouping of roles ({@code Role} objects).
- Role - The base interface for {@code Role} objects managed by the User Admin service.
- User - A {@code User} role managed by a User Admin service.
- UserAdmin - This interface is used to manage a database of named {@code Role} objects, which can be used for authentication and authorization purposes.
- UserAdminEvent - {@code Role} change event.
- UserAdminListener - Listener for UserAdminEvents.
- UserAdminPermission - Permission to configure and access the {@code Role} objects managed by a User Admin service.

107.8.2 public abstract Authorization

The {@code Authorization} interface encapsulates an authorization context on which bundles can base authorization decisions, where appropriate.

Bundles associate the privilege to access restricted resources or operations with roles. Before granting access to a restricted resource or operation, a bundle will check if the {@code Authorization} object passed to it possess the required role, by calling its {@code hasRole} method.

Authorization contexts are instantiated by calling the {@code UserAdmin#getAuthorization(User)} method.

Trusting Authorization objects

There are no restrictions regarding the creation of {@code Authorization} objects. Hence, a service must only accept {@code Authorization} objects from bundles that has been authorized to use the service using code based (or Java 2) permissions.

In some cases it is useful to use {@code ServicePermission} to do the code based access control. A service basing user access control on {@code Authorization} objects passed to it, will then require that a calling bundle has the {@code ServicePermission} to get the service in question. This is the most convenient way. The OSGi environment will do the code based permission check when the calling bundle attempts to get the service from the service registry.

Example: A servlet using a service on a user's behalf. The bundle with the servlet must be given the {@code ServicePermission} to get the Http Service.

However, in some cases the code based permission checks need to be more fine-grained. A service might allow all bundles to get it, but require certain code based permissions for some of its methods.

Example: A servlet using a service on a user's behalf, where some service functionality is open to anyone, and some is restricted by code based permissions. When a restricted method is called (e.g., one handing over an {@code Authorization} object), the service explicitly checks that the calling bundle has permission to make the call.

No Implement Consumers of this API must not implement this interface

107.8.2.1 **public abstract String getName()**

- Gets the name of the {@link User} that this {@code Authorization} context was created for.

Returns The name of the {@link User} object that this {@code Authorization} context was created for, or {@code null} if no user was specified when this {@code Authorization} context was created.

107.8.2.2 **public abstract String[][] getRoles()**

- Gets the names of all roles implied by this {@code Authorization} context.

Returns The names of all roles implied by this {@code Authorization} context, or {@code null} if no roles are in the context. The predefined role {@code user.anyone} will not be included in this list.

107.8.2.3 **public abstract boolean hasRole(String name)**

name The name of the role to check for.

- Checks if the role with the specified name is implied by this {@code Authorization} context.

Bundles must define globally unique role names that are associated with the privilege of accessing restricted resources or operations. Operators will grant users access to these resources, by creating a {@link Group} object for each role and adding {@link User} objects to it.

Returns {@code true} if this {@code Authorization} context implies the specified role, otherwise {@code false}.

107.8.3 **public abstract Group extends User**

A named grouping of roles ({@code Role} objects).

Whether or not a given {@code Authorization} context implies a {@code Group} object depends on the members of that {@code Group} object.

A {@code Group} object can have two kinds of members: *basic* and *required*. A {@code Group} object is implied by an {@code Authorization} context if all of its required members are implied and at least one of its basic members is implied.

A {@code Group} object must contain at least one basic member in order to be implied. In other words, a {@code Group} object without any basic member roles is never implied by any {@code Authorization} context.

A {@code User} object always implies itself.

No loop detection is performed when adding members to {@code Group} objects, which means that it is possible to create circular implications. Loop detection is instead done when roles are checked. The semantics is that if a role depends on itself (i.e., there is an implication loop), the role is not implied.

The rule that a {@code Group} object must have at least one basic member to be implied is motivated by the following example:

```
group foo
  required members: marketing
  basic members: alice, bob
```

Privileged operations that require membership in "foo" can be performed only by "alice" and "bob", who are in marketing.

If "alice" and "bob" ever transfer to a different department, anybody in marketing will be able to assume the "foo" role, which certainly must be prevented. Requiring that "foo" (or any {@code Group} object for that matter) must have at least one basic member accomplishes that.

However, this would make it impossible for a {@code Group} object to be implied by just its required members. An example where this implication might be useful is the following declaration: "Any citizen who is an adult is allowed to vote." An intuitive configuration of "voter" would be:

```
group voter
  required members: citizen, adult
  basic members:
```

However, according to the above rule, the "voter" role could never be assumed by anybody, since it lacks any basic members. In order to address this issue a predefined role named "user.anyone" can be specified, which is always implied. The desired implication of the "voter" group can then be achieved by specifying "user.anyone" as its basic member, as follows:

```
group voter
  required members: citizen, adult
  basic members: user.anyone
```

No Implement Consumers of this API must not implement this interface

107.8.3.1 **public abstract boolean addMember(Role role)**

role The role to add as a basic member.

- Adds the specified {@code Role} object as a basic member to this {@code Group} object.

Returns {@code true} if the given role could be added as a basic member, and {@code false} if this {@code Group} object already contains a {@code Role} object whose name matches that of the specified role.

Throws `SecurityException`— If a security manager exists and the caller does not have the {@code UserAdminPermission} with name {@code admin}.

107.8.3.2 **public abstract boolean addRequiredMember(Role role)**

role The {@code Role} object to add as a required member.

- Adds the specified {@code Role} object as a required member to this {@code Group} object.

Returns {@code true} if the given {@code Role} object could be added as a required member, and {@code false} if this {@code Group} object already contains a {@code Role} object whose name matches that of the specified role.

Throws `SecurityException`— If a security manager exists and the caller does not have the {@code UserAdminPermission} with name {@code admin}.

107.8.3.3 **public abstract Role[] getMembers()**

- Gets the basic members of this {@code Group} object.

Returns The basic members of this {@code Group} object, or {@code null} if this {@code Group} object does not contain any basic members.

107.8.3.4 **public abstract Role[] getRequiredMembers()**

- Gets the required members of this {@code Group} object.

Returns The required members of this {@code Group} object, or {@code null} if this {@code Group} object does not contain any required members.

107.8.3.5 **public abstract boolean removeMember(Role role)**

role The {@code Role} object to remove from this {@code Group} object.

- Removes the specified {@code Role} object from this {@code Group} object.

Returns {@code true} if the {@code Role} object could be removed, otherwise {@code false}.

Throws `SecurityException`—If a security manager exists and the caller does not have the {@code UserAdminPermission} with name {@code admin}.

107.8.4 **public abstract Role**

The base interface for {@code Role} objects managed by the User Admin service.

This interface exposes the characteristics shared by all {@code Role} classes: a name, a type, and a set of properties.

Properties represent public information about the {@code Role} object that can be read by anyone. Specific {@code UserAdminPermission} objects are required to change a {@code Role} object's properties.

{@code Role} object properties are {@code Dictionary} objects. Changes to these objects are propagated to the User Admin service and made persistent.

Every User Admin service contains a set of predefined {@code Role} objects that are always present and cannot be removed. All predefined {@code Role} objects are of type {@code ROLE}. This version of the `org.osgi.service.useradmin` package defines a single predefined role named "user.anyone", which is inherited by any other role. Other predefined roles may be added in the future. Since "user.anyone" is a {@code Role} object that has properties associated with it that can be read and modified. Access to these properties and their use is application specific and is controlled using {@code UserAdminPermission} in the same way that properties for other {@code Role} objects are.

No Implement Consumers of this API must not implement this interface

107.8.4.1 **public static final int GROUP = 2**

The type of a {@code Group} role.

The value of {@code GROUP} is 2.

107.8.4.2 **public static final int ROLE = 0**

The type of a predefined role.

The value of {@code ROLE} is 0.

107.8.4.3 **public static final int USER = 1**

The type of a {@code User} role.

The value of {@code USER} is 1.

107.8.4.4 **public static final String USER_ANYONE = "user.anyone"**

The name of the predefined role, user.anyone, that all users and groups belong to.

Since @since 1.1

107.8.4.5 **public abstract String getName()**

- Returns the name of this role.

Returns The role's name.

107.8.4.6 public abstract Dictionary<String, Object> getProperties()

- Returns a {@code Dictionary} of the (public) properties of this {@code Role} object. Any changes to the returned {@code Dictionary} will change the properties of this {@code Role} object. This will cause a {@code UserAdminEvent} object of type {@link UserAdminEvent#ROLE_CHANGED} to be broadcast to any {@code UserAdminListener} objects.

Only objects of type {@code String} may be used as property keys, and only objects of type {@code String} or {@code byte[]} may be used as property values. Any other types will cause an exception of type {@code IllegalArgumentException} to be raised.

In order to add, change, or remove a property in the returned {@code Dictionary}, a {@link UserAdminPermission} named after the property name (or a prefix of it) with action {@code changeProperty} is required.

Returns {@code Dictionary} containing the properties of this {@code Role} object.

107.8.4.7 public abstract int getType()

- Returns the type of this role.

Returns The role's type.

**107.8.5 public abstract User
extends Role**

A {@code User} role managed by a User Admin service.

In this context, the term "user" is not limited to just human beings. Instead, it refers to any entity that may have any number of credentials associated with it that it may use to authenticate itself.

In general, {@code User} objects are associated with a specific User Admin service (namely the one that created them), and cannot be used with other User Admin services.

A {@code User} object may have credentials (and properties, inherited from the {@link Role} class) associated with it. Specific {@link UserAdminPermission} objects are required to read or change a {@code User} object's credentials.

Credentials are {@code Dictionary} objects and have semantics that are similar to the properties in the {@code Role} class.

No Implement Consumers of this API must not implement this interface

107.8.5.1 public abstract Dictionary<String, Object> getCredentials()

- Returns a {@code Dictionary} of the credentials of this {@code User} object. Any changes to the returned {@code Dictionary} object will change the credentials of this {@code User} object. This will cause a {@code UserAdminEvent} object of type {@link UserAdminEvent#ROLE_CHANGED} to be broadcast to any {@code UserAdminListeners} objects.

Only objects of type {@code String} may be used as credential keys, and only objects of type {@code String} or of type {@code byte[]} may be used as credential values. Any other types will cause an exception of type {@code IllegalArgumentException} to be raised.

In order to retrieve a credential from the returned {@code Dictionary} object, a {@link UserAdminPermission} named after the credential name (or a prefix of it) with action {@code getCredential} is required.

In order to add or remove a credential from the returned {@code Dictionary} object, a {@link UserAdminPermission} named after the credential name (or a prefix of it) with action {@code changeCredential} is required.

Returns {@code Dictionary} object containing the credentials of this {@code User} object.

107.8.5.2 public abstract boolean hasCredential(String key, Object value)

key The credential {@code key}.

value The credential {@code value}.

- Checks to see if this {@code User} object has a credential with the specified {@code key} set to the specified {@code value}.

If the specified credential {@code value} is not of type {@code String} or {@code byte[]}, it is ignored, that is, {@code false} is returned (as opposed to an {@code IllegalArgumentException} being raised).

Returns {@code true} if this user has the specified credential; {@code false} otherwise.

Throws `SecurityException` – If a security manager exists and the caller does not have the {@code UserAdminPermission} named after the credential key (or a prefix of it) with action {@code getCredential}.

107.8.6 public abstract UserAdmin

This interface is used to manage a database of named {@code Role} objects, which can be used for authentication and authorization purposes.

This version of the User Admin service defines two types of {@code Role} objects: "User" and "Group". Each type of role is represented by an {@code int} constant and an interface. The range of positive integers is reserved for new types of roles that may be added in the future. When defining proprietary role types, negative constant values must be used.

Every role has a name and a type.

A {@link User} object can be configured with credentials (e.g., a password) and properties (e.g., a street address, phone number, etc.).

A {@link Group} object represents an aggregation of {@link User} and {@link Group} objects. In other words, the members of a {@code Group} object are roles themselves.

Every User Admin service manages and maintains its own namespace of {@code Role} objects, in which each {@code Role} object has a unique name.

No Implement Consumers of this API must not implement this interface

107.8.6.1 public abstract Role createRole(String name, int type)

name The {@code name} of the {@code Role} object to create.

type The type of the {@code Role} object to create. Must be either a {@link Role#USER} type or {@link Role#GROUP} type.

- Creates a {@code Role} object with the given name and of the given type.

If a {@code Role} object was created, a {@code UserAdminEvent} object of type {@link UserAdminEvent#ROLE_CREATED} is broadcast to any {@code UserAdminListener} object.

Returns The newly created {@code Role} object, or {@code null} if a role with the given name already exists.

Throws `IllegalArgumentException` – if {@code type} is invalid.

`SecurityException` – If a security manager exists and the caller does not have the {@code UserAdminPermission} with name {@code admin}.

107.8.6.2 public abstract Authorization getAuthorization(User user)

user The {@code User} object to create an {@code Authorization} object for, or {@code null} for the anonymous user.

- Creates an {@code Authorization} object that encapsulates the specified {@code User} object and the {@code Role} objects it possesses. The {@code null} user is interpreted as the anonymous user.

The anonymous user represents a user that has not been authenticated. An {@code Authorization} object for an anonymous user will be unnamed, and will only imply groups that user.anyone implies.

Returns the {@code Authorization} object for the specified {@code User} object.

107.8.6.3 **public abstract Role getRole(String name)**

name The name of the {@code Role} object to get.

- Gets the {@code Role} object with the given {@code name} from this User Admin service.

Returns The requested {@code Role} object, or {@code null} if this User Admin service does not have a {@code Role} object with the given {@code name}.

107.8.6.4 **public abstract Role[] getRoles(String filter) throws InvalidSyntaxException**

filter The filter criteria to match.

- Gets the {@code Role} objects managed by this User Admin service that have properties matching the specified LDAP filter criteria. See {@code org.osgi.framework.Filter} for a description of the filter syntax. If a {@code null} filter is specified, all Role objects managed by this User Admin service are returned.

Returns The {@code Role} objects managed by this User Admin service whose properties match the specified filter criteria, or all {@code Role} objects if a {@code null} filter is specified. If no roles match the filter, {@code null} will be returned.

Throws InvalidSyntaxException – If the filter is not well formed.

107.8.6.5 **public abstract User getUser(String key, String value)**

key The property key to look for.

value The property value to compare with.

- Gets the user with the given property {@code key}-{@code value} pair from the User Admin service database. This is a convenience method for retrieving a {@code User} object based on a property for which every {@code User} object is supposed to have a unique value (within the scope of this User Admin service), such as for example a X.500 distinguished name.

Returns A matching user, if *exactly* one is found. If zero or more than one matching users are found, {@code null} is returned.

107.8.6.6 **public abstract boolean removeRole(String name)**

name The name of the {@code Role} object to remove.

- Removes the {@code Role} object with the given name from this User Admin service and all groups it is a member of.

If the {@code Role} object was removed, a {@code UserAdminEvent} object of type {@code UserAdminEvent#ROLE_REMOVED} is broadcast to any {@code UserAdminListener} object.

Returns {@code true} If a {@code Role} object with the given name is present in this User Admin service and could be removed, otherwise {@code false}.

Throws SecurityException – If a security manager exists and the caller does not have the {@code UserAdminPermission} with name {@code admin}.

107.8.7 **public class UserAdminEvent**

{@code Role} change event.

{@code UserAdminEvent} objects are delivered asynchronously to any {@code UserAdminListener} objects when a change occurs in any of the {@code Role} objects managed by a User Admin service.

A type code is used to identify the event. The following event types are defined: {@link #ROLE_CREATED} type, {@link #ROLE_CHANGED} type, and {@link #ROLE_REMOVED} type. Additional event types may be defined in the future.

See Also UserAdmin, UserAdminListener

107.8.7.1 **public static final int ROLE_CHANGED = 2**

A {@code Role} object has been modified.

The value of {@code ROLE_CHANGED} is 0x00000002.

107.8.7.2 **public static final int ROLE_CREATED = 1**

A {@code Role} object has been created.

The value of {@code ROLE_CREATED} is 0x00000001.

107.8.7.3 **public static final int ROLE_REMOVED = 4**

A {@code Role} object has been removed.

The value of {@code ROLE_REMOVED} is 0x00000004.

107.8.7.4 **public UserAdminEvent(ServiceReference<UserAdmin> ref, int type, Role role)**

ref The {@code ServiceReference} object of the User Admin service that generated this event.

type The event type.

role The {@code Role} object on which this event occurred.

- Constructs a {@code UserAdminEvent} object from the given {@code ServiceReference} object, event type, and {@code Role} object.

107.8.7.5 **public Role getRole()**

- Gets the {@code Role} object this event was generated for.

Returns The {@code Role} object this event was generated for.

107.8.7.6 **public ServiceReference<UserAdmin> getServiceReference()**

- Gets the {@code ServiceReference} object of the User Admin service that generated this event.

Returns The User Admin service's {@code ServiceReference} object.

107.8.7.7 **public int getType()**

- Returns the type of this event.

The type values are {@link #ROLE_CREATED} type, {@link #ROLE_CHANGED} type, and {@link #ROLE_REMOVED} type.

Returns The event type.

107.8.8 **public abstract UserAdminListener**

Listener for UserAdminEvents.

{@code UserAdminListener} objects are registered with the Framework service registry and notified with a {@code UserAdminEvent} object when a {@code Role} object has been created, removed, or modified.

{@code UserAdminListener} objects can further inspect the received {@code UserAdminEvent} object to determine its type, the {@code Role} object it occurred on, and the User Admin service that generated it.

See Also UserAdmin, UserAdminEvent

107.8.8.1 public abstract void roleChanged(UserAdminEvent event)

event The {@code UserAdminEvent} object.

- Receives notification that a {@code Role} object has been created, removed, or modified.

107.8.9 public final class UserAdminPermission extends BasicPermission

Permission to configure and access the {@link Role} objects managed by a User Admin service.

This class represents access to the {@code Role} objects managed by a User Admin service and their properties and credentials (in the case of {@link User} objects).

The permission name is the name (or name prefix) of a property or credential. The naming convention follows the hierarchical property naming convention. Also, an asterisk may appear at the end of the name, following a ".", or by itself, to signify a wildcard match. For example: "org.osgi.security.protocol.*" or "*" is valid, but "*protocol" or "a*b" are not valid.

The {@code UserAdminPermission} with the reserved name "admin" represents the permission required for creating and removing {@code Role} objects in the User Admin service, as well as adding and removing members in a {@code Group} object. This {@code UserAdminPermission} does not have any actions associated with it.

The actions to be granted are passed to the constructor in a string containing a list of one or more comma-separated keywords. The possible keywords are: {@code changeProperty}, {@code changeCredential}, and {@code getCredential}. Their meaning is defined as follows:

action	
changeProperty	Permission to change (i.e., add and remove) Role object properties whose names start with the name argument specified in the constructor.
changeCredential	Permission to change (i.e., add and remove) User object credentials whose names start with the name argument specified in the constructor.
getCredential	Permission to retrieve and check for the existence of User object credentials whose names start with the name argument specified in the constructor.

The action string is converted to lowercase before processing.

Following is a PermissionInfo style policy entry which grants a user administration bundle a number of {@code UserAdminPermission} object:

```
(org.osgi.service.useradmin.UserAdminPermission "admin")
(org.osgi.service.useradmin.UserAdminPermission "com.foo.*"
 "changeProperty,getCredential,changeCredential")
(org.osgi.service.useradmin.UserAdminPermission "user.*"
 "changeProperty,changeCredential")
```

The first permission statement grants the bundle the permission to perform any User Admin service operations of type "admin", that is, create and remove roles and configure {@code Group} objects.

The second permission statement grants the bundle the permission to change any properties as well as get and change any credentials whose names start with {@code com.foo.}.

The third permission statement grants the bundle the permission to change any properties and credentials whose names start with {@code user.}. This means that the bundle is allowed to change, but not retrieve any credentials with the given prefix.

The following policy entry empowers the Http Service bundle to perform user authentication:

```
grant codeBase "${jars}http.jar" {
    permission org.osgi.service.useradmin.UserAdminPermission
        "user.password", "getCredential";
};
```

The permission statement grants the Http Service bundle the permission to validate any password credentials (for authentication purposes), but the bundle is not allowed to change any properties or credentials.

Concurrency Thread-safe

107.8.9.1 **public static final String ADMIN = "admin"**

The permission name "admin".

107.8.9.2 **public static final String CHANGE_CREDENTIAL = "changeCredential"**

The action string "changeCredential".

107.8.9.3 **public static final String CHANGE_PROPERTY = "changeProperty"**

The action string "changeProperty".

107.8.9.4 **public static final String GET_CREDENTIAL = "getCredential"**

The action string "getCredential".

107.8.9.5 **public UserAdminPermission(String name, String actions)**

name the name of this {@code UserAdminPermission}

actions the action string.

- Creates a new {@code UserAdminPermission} with the specified name and actions. {@code name} is either the reserved string "admin" or the name of a credential or property, and {@code actions} contains a comma-separated list of the actions granted on the specified name. Valid actions are {@code changeProperty},{@code changeCredential}, and `getCredential`.

Throws `IllegalArgumentException` – If {@code name} equals "admin" and {@code actions} are specified.

107.8.9.6 **public boolean equals(Object obj)**

obj the object to be compared for equality with this object.

- Checks two {@code UserAdminPermission} objects for equality. Checks that {@code obj} is a {@code UserAdminPermission}, and has the same name and actions as this object.

Returns {@code true} if {@code obj} is a {@code UserAdminPermission} object, and has the same name and actions as this {@code UserAdminPermission} object.

107.8.9.7 **public String getActions()**

- Returns the canonical string representation of the actions, separated by comma.

Returns the canonical string representation of the actions.

107.8.9.8 **public int hashCode()**

- Returns the hash code value for this object.

Returns A hash code value for this object.

107.8.9.9 **public boolean implies(Permission p)**

p the permission to check against.

- Checks if this {@code UserAdminPermission} object "implies" the specified permission.

More specifically, this method returns `{@code true}` if:

- *p* is an instanceof `{@code UserAdminPermission}`,
- *p*'s actions are a proper subset of this object's actions, and
- *p*'s name is implied by this object's name. For example, "java.*" implies "java.home".

Returns `{@code true}` if the specified permission is implied by this object; `{@code false}` otherwise.

107.8.9.10 **public PermissionCollection newPermissionCollection()**

- Returns a new `{@code PermissionCollection}` object for storing `{@code UserAdminPermission}` objects.

Returns a new `{@code PermissionCollection}` object suitable for storing `{@code UserAdminPermission}` objects.

107.8.9.11 **public String toString()**

- Returns a string describing this `{@code UserAdminPermission}` object. This string must be in `{@code PermissionInfo}` encoded format.

Returns The `{@code PermissionInfo}` encoded string for this `{@code UserAdminPermission}` object.

See Also `org.osgi.service.permissionadmin.PermissionInfo.getEncoded()`

107.9 References

- [1] *The Java Security Architecture for JDK 1.2*
Version 1.0, Sun Microsystems, October 1998
- [2] *Java Authentication and Authorization Service*
<https://docs.oracle.com/javase/8/docs/technotes/guides/security/jaas/JAASRefGuide.html>

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT